PHASE 3 — DEEP DIVE

Python for AI Engineering

Month 6–8 · Every resource with exact video title + channel name

AI Product Engineer Master Roadmap — Phase 3 Companion Document

Phase Duration	12 weeks (Month 6, 7, and 8 of the full roadmap)
Daily Commitment	2–3 hours weekdays, 4–6 hours weekends
Your Starting Point	Phase 2 complete — you have a full-stack AI SaaS backend in production. Now add Python.
Goal of This Phase	Become fluent in Python for AI — pipelines, data processing, FastAPI microservices, async patterns
Why Python for AI	Every AI/ML library (LangChain, HuggingFace, LlamaIndex, vLLM, Prefect) is Python-first. You cannot build real AI without it.
Total Topics	9 core topics — Python fundamentals, env management, NumPy, Pandas, data viz, file parsing, FastAPI, async Python, data pipelines
JS Bridge Strategy	You are NOT abandoning JavaScript. Python handles AI/ML logic as a microservice. Next.js calls the Python FastAPI. Both live together.
Capstone Project	AI Document Intelligence Pipeline — Portfolio Project #3

⍰ Why Python? The JS Developer's Mindset Shift

You are a JavaScript developer. Python feels foreign. That is normal and temporary. Here is the most important framing for this phase: you are not replacing JavaScript with Python — you are adding Python as a second language for a specific purpose. JavaScript runs your frontend and API. Python runs your AI/ML logic. They communicate via HTTP.

JavaScript — Keep Using It For:	Python — Use It For:
Frontend (React, Next.js)	ML libraries (LangChain, HuggingFace, vLLM)
REST API / BFF layer (Fastify)	FastAPI AI microservices
Vercel AI SDK streaming	Data processing pipelines
Stripe, Clerk, Postgres (Prisma)	NumPy / Pandas data manipulation
Next.js server actions	Document parsing (PyMuPDF, python-docx)
Real-time features (SSE, WebSockets)	Embeddings generation at scale
CI/CD, deployment scripts	Model fine-tuning and inference
User-facing SaaS application	Batch AI processing jobs

⍰ **The Architecture:** Next.js (frontend) → Fastify (main API) → Python FastAPI (AI microservice). This is how every serious AI startup structures their stack. Your Node.js skills are not wasted — they power the user-facing product. Python powers the AI engine.

TOPIC 1 — Python Fundamentals & Pythonic Patterns · *Weeks 1–2 · Go from zero to confident in Python*

You already know programming concepts — loops, functions, classes, data structures. Python just has different syntax and idioms. The goal here is NOT to memorise every Python feature — it is to become fluent enough to read Python AI code confidently and write Python scripts without constantly consulting documentation.

1.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Python for Beginners — Full Course (4.5 hrs)	Programming with Mosh	YouTube → Search 'Python for Beginners Full Course Programming with Mosh'	Best Python intro course for developers who already know another language. Fast-paced, no fluff.
2	Course	Python Tutorial — Python Full Course for Beginners (6 hrs)	Tech With Tim	YouTube → Search 'Python Full Course Beginners Tech With Tim 6 hours'	More detailed alternative to Mosh. Good for concepts that need extra explanation.
3	Video	Python in 100 Seconds	Fireship	YouTube → Search 'Python in 100 Seconds Fireship'	Watch first — 2-min orientation. Sets tone before diving into full courses.
4	Playlist	Intermediate Python Programming Course (25 videos)	Tech With Tim	YouTube → Search 'Intermediate Python Programming Course Tech With Tim' → open playlist	After basics: covers decorators, generators, context managers, itertools, functools.
5	Video	Python Decorators — How They Actually Work	Corey Schafer	YouTube → Search 'Python Decorators How They Actually Work Corey Schafer'	Decorators are everywhere in Python AI libraries. This is the best explanation.
6	Video	Python Generators — How They Work and Why to Use Them	Corey Schafer	YouTube → Search 'Python Generators How They Work Corey Schafer'	Generators power lazy data loading in ML — critical for large dataset handling.
7	Video	Python Context Managers — with Statement Explained	Corey Schafer	YouTube → Search 'Python Context Managers with Statement Corey Schafer'	Used everywhere in file handling, database sessions, model loading.
8	Video	Python List Comprehensions, Dict Comprehensions, Generator Expressions	Corey Schafer	YouTube → Search 'Python List Comprehensions Corey Schafer'	Pythonic data transformations — used constantly in data preprocessing pipelines.
9	Video	Python OOP Tutorial — Classes and Objects (6-part series)	Corey Schafer	YouTube → Search 'Python OOP Tutorial Corey Schafer' → open playlist (6 videos)	Python OOP is simpler than JS but has key differences (dunder methods, dataclasses).

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
10	Video	Python Type Hints — Full Guide (mypy, typing module)	ArjanCodes	YouTube → Search 'Python Type Hints Full Guide ArjanCodes'	Type hints make Python as safe as TypeScript. Essential for production AI code.
11	Website	python-tutorial.eu — Complete Python Reference	Wolfgang Gruber	Visit python-tutorial.eu → read all sections	Best written Python reference for developers coming from other languages.
12	Book	Automate the Boring Stuff with Python (free online)	Al Sweigart	automatetheboringstuff.com — free to read online	Free book. Chapters 1–8 solidify fundamentals through practical projects.

1.2 — Python vs JavaScript: Key Differences

Syntax Differences to Know Immediately

```
# Python — no braces, use indentation
def process_document(file_path: str, max_chunks: int = 50) -> list[str]:
    """Process a document and return text chunks."""
    with open(file_path, 'r') as f:      # context manager (like try/finally)
        content = f.read()
    chunks = [content[i:i+512]           # list comprehension (like .map())
              for i in range(0, len(content), 512)]
    return chunks[:max_chunks]

# Equivalent JavaScript would be:
// function processDocument(filePath, maxChunks = 50) {
//   const content = fs.readFileSync(filePath, 'utf8');
//   const chunks = Array.from({length: Math.ceil(content.length/512)},
//     (_, i) => content.slice(i*512, i*512+512));
//   return chunks.slice(0, maxChunks);
// }
```

Critical Pythonic Patterns for AI Code

- List comprehensions: [f(x) for x in items if condition(x)] — used in every data pipeline
- Dict comprehensions: {k: v for k, v in pairs} — building metadata maps
- Generator expressions: (f(x) for x in items) — memory-efficient for large datasets
- Unpacking: a, b, *rest = sequence — tuple unpacking in ML result handling
- f-strings: f'Processing {len(docs)} documents...' — preferred string formatting
- Walrus operator: if chunk := get_next_chunk(): — useful in parsing loops
- Dataclasses: @dataclass — lightweight typed data containers for API request/response

Decorators — Seen Constantly in FastAPI & LangChain

- @property — computed attributes (like a getter)
- @classmethod — constructor alternatives
- @staticmethod — utility functions
- @functools.lru_cache — memoize expensive function calls (cache embedding results)
- @retry — from tenacity library — retry failed LLM API calls automatically

> ⍰ The most important decorator to master for AI: @lru_cache. Wrap your embedding function with it to cache vectors for repeated texts. A single line saves thousands of API calls.

1.3 — Stop Conditions (Python Fundamentals)

- ☐ Rewrite a Node.js REST API handler in Python — identical behaviour, idiomatic Python style
- ☐ Write a list comprehension that filters and transforms a list of AI documents in one line
- ☐ Implement a decorator @retry(max_attempts=3, backoff=exponential) for LLM API calls
- ☐ Use dataclasses to define a typed Document, Chunk, and EmbeddingResult data model
- ☐ Implement __iter__ and __next__ on a class to lazily load documents from a directory
- ☐ Explain the difference between a list, tuple, set, and dict — when to use each
- ☐ Write a context manager using __enter__ and __exit__ for a temporary file processor
- ☐ Use type hints throughout — mypy runs with zero errors on your code

TOPIC 2 — Virtual Environments, Poetry & Project Setup · *Week 2 · Broken envs kill days — nail this first*

Python's environment management is the biggest source of frustration for developers new to the language. Packages installed globally conflict with each other. Different projects need different library versions. Poetry solves this cleanly. Spend half a day on this topic — it prevents days of debugging broken installs later.

2.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Python Virtual Environments — Full Guide (venv, pip, pipenv)	Corey Schafer	YouTube → Search 'Python Virtual Environments Full Guide Corey Schafer'	The definitive venv tutorial. Understand the basics before moving to Poetry.
2	Video	Poetry — Python Dependency Management Full Tutorial	ArjanCodes	YouTube → Search 'Poetry Python Dependency Management Full Tutorial ArjanCodes'	Best Poetry tutorial. Covers pyproject.toml, groups, versioning, publishing.
3	Video	Python Project Setup — The Right Way in 2024 (Poetry + Ruff + mypy)	ArjanCodes	YouTube → Search 'Python Project Setup Right Way 2024 ArjanCodes'	Complete modern Python project setup. This is the template for all Phase 3 projects.
4	Video	Ruff — The Fastest Python Linter Explained	ArjanCodes	YouTube → Search 'Ruff Fastest Python Linter ArjanCodes'	Ruff replaces flake8 + isort + black. One tool for all Python formatting and linting.
5	Video	Python .env Files and Environment Variables — Best Practices	Tech With Tim	YouTube → Search 'Python env Files Environment Variables Best Practices Tech With Tim'	python-dotenv and Pydantic Settings for managing API keys and config.
6	Video	Docker for Python Developers — Full Tutorial	Tech With Tim	YouTube → Search 'Docker for Python Developers Full Tutorial Tech With Tim'	Containerise your FastAPI service — needed for deployment in Phase 6.
7	Docs	Poetry Official Documentation	Poetry Team	python-poetry.org/docs — read Basic Usage + Managing Environments	Short, excellent docs. Read the Basic Usage and Managing Environments sections.

2.2 — Standard Project Structure for All Phase 3+ Projects

```
ai-doc-pipeline/              # Project root
├── pyproject.toml            # Poetry config — replaces requirements.txt
├── .env                      # Local secrets (in .gitignore)
├── .env.example              # Template with all variables (committed)
├── Dockerfile                # Container definition
```

```
├── src/
│   └── ai_pipeline/
│       ├── __init__.py
│       ├── main.py           # FastAPI application entry point
│       ├── routers/          # API route handlers
│       │   ├── documents.py  # POST /documents, GET /documents/{id}
│       │   └── health.py     # GET /health
│       ├── services/         # Business logic
│       │   ├── parser.py     # Document parsing logic
│       │   ├── embedder.py   # Embedding generation
│       │   └── pipeline.py   # Orchestration
│       ├── models/           # Pydantic models (request/response schemas)
│       │   └── document.py
│       └── config.py         # Pydantic Settings — loads from .env
└── tests/
    ├── conftest.py           # Pytest fixtures
    └── test_parser.py        # Unit tests
```

Essential pyproject.toml Structure

```
# pyproject.toml — the single config file for everything
[tool.poetry]
name = "ai-pipeline"
version = "0.1.0"
description = "AI document intelligence pipeline"
python = "^3.11"

[tool.poetry.dependencies]
python = "^3.11"
fastapi = "^0.111.0"
uvicorn = {extras = ["standard"], version = "^0.30.0"}
pydantic = "^2.7.0"
pydantic-settings = "^2.3.0"
openai = "^1.35.0"
pymupdf = "^1.24.0"       # PDF parsing
python-docx = "^1.1.0"    # DOCX parsing
httpx = "^0.27.0"         # Async HTTP client
tenacity = "^8.5.0"       # Retry logic

[tool.poetry.group.dev.dependencies]
pytest = "^8.2.0"
pytest-asyncio = "^0.23.0"
ruff = "^0.5.0"
mypy = "^1.10.0"
```

2.3 — Stop Conditions

☐ Create a new Poetry project — 'poetry new ai-pipeline' — with correct directory structure
☐ Add dependencies with 'poetry add', dev dependencies with 'poetry add --group dev'
☐ Set up Ruff linting + formatting — 'poetry run ruff check .' passes with zero errors
☐ Set up mypy type checking — 'poetry run mypy src/' passes with zero errors
☐ Configure Pydantic Settings to load all config from .env file — no os.environ calls in business logic
☐ Docker: build and run your FastAPI project in a container — 'docker compose up' works in <60s

TOPIC 3 — NumPy: Numerical Computing & Tensor Math · *Week 3 · The math layer under every AI library*

NumPy is the foundational numerical computing library. PyTorch tensors, TensorFlow tensors, and even LLM embedding vectors are built on NumPy's ndarray concept. You do not need to be a NumPy expert, but understanding arrays, shapes, broadcasting, and basic linear algebra operations is essential for debugging AI models and understanding embeddings.

3.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	NumPy Full Python Tutorial — Data Science Fundamentals (3.5 hrs)	Keith Galli	YouTube → Search 'NumPy Full Python Tutorial Data Science Keith Galli'	The most comprehensive NumPy tutorial on YouTube. Covers all operations you need.
2	Video	NumPy in 15 Minutes	Mosh Hamedani	YouTube → Search 'NumPy in 15 Minutes Mosh Hamedani'	Fast-paced intro — watch before Keith Galli for orientation.
3	Video	NumPy Array Indexing — Slicing, Masking, Fancy Indexing	Keith Galli	YouTube → Search 'NumPy Array Indexing Slicing Masking Keith Galli'	Indexing operations are used constantly when processing batches of embeddings.
4	Video	NumPy Broadcasting Explained — The Most Confusing NumPy Concept	CS Dojo	YouTube → Search 'NumPy Broadcasting Explained CS Dojo'	Broadcasting is the source of 50% of NumPy confusion. This video fixes it permanently.
5	Video	Linear Algebra for Machine Learning — Full Course (3 hrs)	freeCodeCamp.org	YouTube → Search 'Linear Algebra Machine Learning Full Course freeCodeCamp'	Vectors, matrices, dot products, cosine similarity — the math behind embeddings.
6	Video	Cosine Similarity in Python — How to Compute Embedding Similarity	NeuralNine	YouTube → Search 'Cosine Similarity Python Embedding Similarity NeuralNine'	Directly shows how to compute similarity between embedding vectors — core RAG operation.
7	Docs	NumPy Official Documentation — Quickstart Tutorial	NumPy Team	numpy.org/doc/stable/user/quickstart.html	Best written NumPy reference. Read the full quickstart, then the 'How to' guides.

3.2 — What to Study

Array Creation & Data Types

- np.array(), np.zeros(), np.ones(), np.arange(), np.linspace()
- dtype: float32 vs float64 — ML models use float32 (half the memory)
- Shape inspection: arr.shape, arr.ndim, arr.dtype, arr.size
- Reshaping: arr.reshape(), arr.flatten(), arr.ravel()

Operations on Embedding Vectors (The Core AI Use Case)

- Dot product: np.dot(a, b) — measures vector similarity
- Cosine similarity: np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
- Batch cosine similarity: compute similarity of one vector vs 10,000 stored vectors efficiently
- Normalisation: arr / np.linalg.norm(arr) — normalise embedding to unit sphere
- Top-K: np.argsort(similarities)[-k:] — find the K most similar embeddings

> ⍰ Practical exercise: Generate 10,000 random 1536-dimensional vectors (OpenAI embedding size). Compute cosine similarity of a query vector against all 10,000 in one NumPy operation. Time it — it should complete in <10ms. This is naive vector search.

Broadcasting — Eliminates Loops

- Broadcasting rule: dimensions are compatible if they are equal or one of them is 1
- Example: add a (1536,) vector to each row of a (10000, 1536) matrix — no loop needed
- Practical: apply the same transformation to an entire batch of embeddings at once

3.3 — Stop Conditions

☐ Implement cosine similarity from scratch using NumPy — no sklearn or scipy
☐ Find the top-5 most similar documents to a query from 10,000 embeddings using pure NumPy
☐ Implement batch normalisation of 1000 embedding vectors — shapes are correct, no loops
☐ Explain broadcasting: why np.ones((1000, 1536)) + np.ones((1536,)) works correctly
☐ Reshape a batch of token embeddings from (batch, seq_len, hidden) to (batch, hidden) using mean pooling

TOPIC 4 — Pandas: Data Cleaning & Transformation · *Week 4 · Clean data = quality AI outputs*

Garbage in, garbage out. RAG pipelines start with raw documents — CSVs full of inconsistencies, PDFs with garbled text, databases with null values and duplicates. Pandas is your tool for cleaning, transforming, and inspecting this data before it ever reaches an LLM. It is a core skill for every AI engineer.

4.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Pandas Tutorial — Full Course for Beginners (4 hrs)	Rob Mulla	YouTube → Search 'Pandas Tutorial Full Course Beginners Rob Mulla'	The best Pandas course on YouTube. Rob is known for practical, no-fluff teaching.
2	Course	Pandas — Kaggle Micro-Course (free, 4 hours)	Kaggle (official)	kaggle.com/learn/pandas — free interactive course	Interactive exercises in the browser. Complete this alongside Rob Mulla's videos.
3	Video	Pandas Data Cleaning Tutorial — Real World Dataset	Keith Galli	YouTube → Search 'Pandas Data Cleaning Tutorial Real World Dataset Keith Galli'	Clean a messy real-world dataset. Directly applicable to AI document preprocessing.
4	Video	Pandas GroupBy — Explained with Real Examples	Corey Schafer	YouTube → Search 'Pandas GroupBy Explained Corey Schafer'	GroupBy for aggregating document metadata — count by category, sum by token usage.
5	Video	Pandas Merge, Join, and Concatenate — Full Guide	Corey Schafer	YouTube → Search 'Pandas Merge Join Concatenate Full Guide Corey Schafer'	Joining multiple data sources — combine document metadata with AI processing results.
6	Video	Pandas apply(), map(), applymap() — When to Use Which	Rob Mulla	YouTube → Search 'Pandas apply map applymap Rob Mulla'	Vectorised operations for transforming entire columns — clean text, normalise fields.
7	Video	Pandas Performance — How to Speed Up Your Code (Vectorisation vs Loops)	Rob Mulla	YouTube → Search 'Pandas Performance Speed Up Vectorisation Loops Rob Mulla'	Never use Python loops in Pandas. This video shows the right patterns.
8	Video	Pandas with Large Datasets — Chunking and Memory Optimisation	Rob Mulla	YouTube → Search 'Pandas Large Datasets Chunking Memory Rob Mulla'	Process datasets larger than RAM using chunks — critical for large document corpora.
9	Docs	Pandas Official User Guide	Pandas Team	pandas.pydata.org/docs/user_guide — read Intro, Indexing, GroupBy, Missing Data	Best reference docs. Read the 10 Minutes to Pandas, then Indexing and GroupBy guides.

4.2 — What to Study

Core Data Structures

- Series: 1D labelled array — a single column of data
- DataFrame: 2D table — rows are documents, columns are metadata fields
- Index: the row labels — use document_id as index for fast lookup
- dtypes: object (string), int64, float64, bool, datetime64 — check and convert

Data Cleaning — The AI Engineer's Daily Work

- Missing values: df.isnull().sum() to diagnose, df.dropna() or df.fillna() to fix
- Duplicates: df.duplicated() to detect, df.drop_duplicates() to remove
- Data type conversion: df['tokens'] = df['tokens'].astype(int)
- String cleaning: df['content'].str.strip().str.lower().str.replace() — clean document text
- Outlier detection: df[df['token_count'] > 8000] — find documents that exceed LLM context
- Date parsing: pd.to_datetime(df['created_at']) — normalise timestamp formats

Practical AI Dataset Operations

- Chunking strategy analysis: df.groupby('doc_id')['chunk_token_count'].describe()
- Category distribution: df['category'].value_counts() — class imbalance check
- Export to Parquet: df.to_parquet('docs.parquet') — efficient storage for large datasets
- Load from CSV/JSONL: pd.read_csv(), pd.read_json(lines=True) — common AI dataset formats

> ⍰ Golden workflow for any new AI dataset: (1) df.head(20) to see the data, (2) df.info() to check types and nulls, (3) df.describe() for numeric stats, (4) df.value_counts() on categorical columns. Do this before writing any pipeline code.

4.3 — Stop Conditions

☐ Load a messy 10,000-row CSV of documents — diagnose all data quality issues in <5 minutes
☐ Clean the dataset: remove duplicates, fill missing fields, normalise text encoding, convert types
☐ Write a pipeline function that takes a raw DataFrame and returns a clean one — fully tested
☐ Compute token count distribution across all documents — identify p95 length (relevant for chunking)
☐ Split a 5GB dataset into 500MB chunks and process each chunk — never load the full dataset into memory
☐ Export processed data to Parquet format — verify file size and load speed vs CSV

TOPIC 5 — Data Visualisation (Matplotlib & Plotly) · *Week 4–5 · See your data before feeding it to AI*

Data visualisation is not just for data scientists. As an AI engineer, you need to visualise token distributions, embedding clusters, retrieval quality metrics, and model performance over time. Matplotlib for static analysis charts, Plotly for interactive dashboards in your AI admin panel.

5.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Matplotlib Tutorial — Full Python Tutorial (2.5 hrs)	Keith Galli	YouTube → Search 'Matplotlib Tutorial Full Python Tutorial Keith Galli'	Best comprehensive Matplotlib tutorial. Covers all chart types you will need.
2	Video	Matplotlib in 10 Minutes	Mosh Hamedani	YouTube → Search 'Matplotlib in 10 Minutes Mosh Hamedani'	Quick overview — watch first for orientation.
3	Video	Seaborn Tutorial — Statistical Data Visualisation	Corey Schafer	YouTube → Search 'Seaborn Tutorial Statistical Data Visualisation Corey Schafer'	Seaborn for beautiful statistical plots — distribution charts, heatmaps.
4	Video	Plotly Tutorial — Interactive Python Visualisations (1.5 hrs)	Charming Data	YouTube → Search 'Plotly Tutorial Interactive Python Visualisations Charming Data'	Plotly for interactive charts — embed in your AI admin dashboard.
5	Video	Visualising High-Dimensional Embeddings with t-SNE and UMAP	ritvikmath	YouTube → Search 'Visualising High Dimensional Embeddings t-SNE UMAP ritvikmath'	Visualise embedding clusters to debug RAG quality — essential debugging skill.
6	Video	Python Pandas + Matplotlib — Full Data Analysis Project	Tech With Tim	YouTube → Search 'Python Pandas Matplotlib Full Data Analysis Project Tech With Tim'	End-to-end analysis project — combines Pandas + Matplotlib. Build this alongside watching.

5.2 — AI-Specific Visualisations to Build

- Token count histogram: distribution of document lengths across your corpus
- Embedding cluster scatter plot: t-SNE / UMAP to visualise topic clusters in 2D
- Retrieval quality chart: precision@k vs recall@k across different chunk sizes
- API latency histogram: distribution of LLM response times by model
- Cost over time: daily API spending by model, user, and feature
- Confusion matrix: for classification pipelines (document categorisation)

5.3 — Stop Conditions

☐ Create a 4-panel Matplotlib figure: token distribution, category breakdown, time series, scatter plot
☐ Build a t-SNE visualisation of 1000 document embeddings — clusters are visually distinguishable
☐ Create an interactive Plotly dashboard showing RAG retrieval metrics
☐ Produce a cost analysis chart: daily LLM API spending by model over 30 days

TOPIC 6 — File I/O & Document Parsing · *Week 5–6 · The first step of every RAG pipeline*

Every RAG pipeline starts the same way: parse a document, extract clean text, split into chunks, embed. This topic covers the parsing step for every document type your AI app will encounter: PDF, DOCX, HTML, CSV, JSON, images with OCR. Reliable parsing is the most underrated skill in AI engineering.

6.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	PyMuPDF Tutorial — Extract Text, Tables, Images from PDFs	Nicholas Renotte	YouTube → Search 'PyMuPDF Tutorial Extract Text Tables Images PDFs Nicholas Renotte'	PyMuPDF (fitz) is the fastest and most capable PDF library. This is the one to use.
2	Video	Python PDF Processing — Read, Write, and Extract Data	Corey Schafer	YouTube → Search 'Python PDF Processing Read Write Extract Corey Schafer'	Comprehensive PDF handling. Covers pypdf, PyMuPDF, and edge cases.
3	Video	Python-docx Tutorial — Read and Write Word Documents	Tech With Tim	YouTube → Search 'Python-docx Tutorial Read Write Word Documents Tech With Tim'	python-docx is the standard DOCX library. Covers paragraphs, tables, headers.
4	Video	Beautiful Soup 4 Tutorial — Web Scraping with Python (1.5 hrs)	Tech With Tim	YouTube → Search 'Beautiful Soup 4 Tutorial Web Scraping Python Tech With Tim'	HTML parsing with BeautifulSoup4 — extract text from web pages for RAG.
5	Video	Python CSV and JSON File Handling — Complete Tutorial	Corey Schafer	YouTube → Search 'Python CSV JSON File Handling Complete Tutorial Corey Schafer'	CSV and JSON are the most common structured data formats. Know both cold.
6	Video	Tesseract OCR with Python — Extract Text from Images	Tech With Tim	YouTube → Search 'Tesseract OCR Python Extract Text from Images Tech With Tim'	OCR for processing scanned PDFs and images — needed for real-world document AI.
7	Video	Unstructured.io — Document Parsing for LLM Applications	Unstructured (official channel)	YouTube → Search 'Unstructured.io Document Parsing LLM Applications'	Unstructured is the all-in-one document parsing library used in production RAG systems.
8	Video	Extract Tables from PDFs with Python — Camelot and pdfplumber	Coding is Fun	YouTube → Search 'Extract Tables PDFs Python Camelot pdfplumber Coding is Fun'	Table extraction is the hardest part of PDF parsing — critical for financial documents.
9	Docs	PyMuPDF Documentation	Artifex Software	pymupdf.readthedocs.io — read Getting Started + Recipes	Best PDF library docs. The Recipes section has code for every use case.

6.2 — Document Parsing Architecture

The Universal Document Parser Design

```
# Universal document parser — handles all formats
from pathlib import Path
from typing import Protocol

class ParsedDocument:
    text: str
    metadata: dict       # {title, author, pages, language, source_url}
    tables: list[list]   # extracted tables as lists
    images: list[bytes]  # extracted images for multi-modal processing

class DocumentParser(Protocol):
    def parse(self, file_path: Path) -> ParsedDocument: ...

class PDFParser:          # uses PyMuPDF (fitz)
    def parse(self, file_path)  -> ParsedDocument: ...

class DOCXParser:          # uses python-docx
    def parse(self, file_path) -> ParsedDocument: ...

class HTMLParser:          # uses BeautifulSoup4
    def parse(self, file_path) -> ParsedDocument: ...

PARSERS = {'.pdf': PDFParser, '.docx': DOCXParser, '.html': HTMLParser}

def parse_document(file_path: str) -> ParsedDocument:
    ext = Path(file_path).suffix.lower()
    parser = PARSERS.get(ext)                  # dispatch by extension
    if not parser:
        raise ValueError(f'Unsupported: {ext}')
    return parser().parse(Path(file_path))
```

Text Chunking Strategies for RAG

- Fixed-size chunking: split every N characters — simple but ignores semantics
- Sentence chunking: split on sentence boundaries — better context preservation
- Semantic chunking: use embedding similarity to find natural break points
- Sliding window: N tokens with K overlap — prevents losing context at boundaries
- Recursive character text splitter (LangChain): tries \n\n, then \n, then ' ' in order

> ⍰ The sliding window approach (512 tokens, 50 token overlap) is the best starting point for most RAG systems. The overlap prevents answers from being split across chunk boundaries.

Metadata Extraction — Critical for Filtered RAG

- Source URL / file path — for citations in AI responses
- Page numbers — reference the exact page in source documents
- Section headers — extracted from PDF bookmarks or DOCX styles
- Creation date — for time-filtered queries ('documents from last year')
- Document type — filter by type (contract, invoice, report) in vector DB queries

6.3 — Stop Conditions

☐ Build a universal parser that handles PDF, DOCX, HTML, TXT with a single function call
☐ Extract tables from a financial PDF — tables returned as Python lists of lists
☐ Apply OCR to a scanned PDF (no selectable text) — readable text extracted correctly
☐ Implement sliding window chunking (512 tokens, 50 overlap) with correct metadata tracking
☐ Process a 500-page PDF — extract text, split into chunks, output JSONL file — all in <30 seconds
☐ Handle parsing errors gracefully: corrupt PDF, password-protected file, empty document

TOPIC 7 — FastAPI: Building AI Microservices · *Week 6–7 · The Python web framework for AI engineers*

FastAPI is arguably the best-documented web framework in any language. It is built on Pydantic for automatic validation and OpenAPI for automatic documentation. It is fully async, type-safe, and 2-3x faster than Flask. Your Node.js Fastify backend will call your Python FastAPI service via HTTP — this is the standard AI startup architecture.

7.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	FastAPI Tutorial — Build a REST API with Python (3 hrs)	ArjanCodes	YouTube → Search 'FastAPI Tutorial Build REST API Python ArjanCodes'	Best FastAPI course. ArjanCodes teaches production patterns, not just hello-world.
2	Course	FastAPI Full Course for Beginners (3.5 hrs)	freeCodeCamp.org	YouTube → Search 'FastAPI Full Course Beginners freeCodeCamp'	Comprehensive intro — covers authentication, database, background tasks, testing.
3	Video	FastAPI + SQLAlchemy + PostgreSQL — Full Project Tutorial	Eric Roby	YouTube → Search 'FastAPI SQLAlchemy PostgreSQL Full Project Tutorial Eric Roby'	Connect FastAPI to PostgreSQL with SQLAlchemy — the async DB stack for Python AI services.
4	Video	Pydantic v2 — The Complete Guide	ArjanCodes	YouTube → Search 'Pydantic v2 Complete Guide ArjanCodes'	Pydantic v2 is the foundation of FastAPI. Validators, models, serialisation — all here.
5	Video	FastAPI Background Tasks — Async Task Processing	Tech With Tim	YouTube → Search 'FastAPI Background Tasks Async Task Processing Tech With Tim'	FastAPI's built-in background tasks for simple async AI operations.
6	Video	FastAPI Dependency Injection — How to Use It Properly	ArjanCodes	YouTube → Search 'FastAPI Dependency Injection How to Use It ArjanCodes'	Dependency injection for database sessions, AI clients, config — the right pattern.
7	Video	FastAPI Streaming Responses — Stream LLM Output in Python	Pixegami	YouTube → Search 'FastAPI Streaming Responses Stream LLM Output Pixegami'	Directly relevant: stream OpenAI tokens from FastAPI to your Node.js backend.
8	Video	FastAPI Testing — Full Guide with pytest and httpx	ArjanCodes	YouTube → Search 'FastAPI Testing Full Guide pytest httpx ArjanCodes'	Testing FastAPI endpoints with pytest + httpx.AsyncClient — mock all AI dependencies.
9	Video	FastAPI Docker Deployment — Production Setup	Tech With Tim	YouTube → Search 'FastAPI Docker Deployment Production Setup Tech With Tim'	Containerise FastAPI for deployment on Railway, Fly.io, or AWS ECS.

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
10	Docs	FastAPI Official Documentation (the best docs in any framework)	Sebastián Ramírez	fastapi.tiangolo.com — read Tutorial, Advanced User Guide completely	Read the entire Tutorial first, then the Advanced section. These docs are exceptionally good.

7.2 — What to Study

FastAPI Core Features

- Path operations: @app.get(), @app.post(), @app.put(), @app.delete()
- Path parameters, query parameters, request body — all typed with Pydantic
- Response models: define exact shape of API response — automatic serialisation
- Status codes: response_model + status_code parameters
- Automatic OpenAPI documentation at /docs (Swagger) and /redoc

Pydantic Models — The Heart of FastAPI

- BaseModel: define request and response schemas with type annotations
- Field validators: @field_validator — validate document content, sanitise inputs
- Model configuration: model_config = ConfigDict(...) for Pydantic v2
- Nested models: DocumentRequest contains EmbeddingConfig contains ModelSettings
- Computed fields: @computed_field for derived properties like token_count

AI Service Patterns in FastAPI

```
# FastAPI AI microservice pattern
from fastapi import FastAPI, Depends, HTTPException
from pydantic import BaseModel
from openai import AsyncOpenAI
import httpx

class DocumentRequest(BaseModel):
    content: str              # validated: min 1 char, max 50000 chars
    model: str = 'gpt-4o'
    extract_schema: dict | None = None

class DocumentResponse(BaseModel):
    extracted: dict
    tokens_used: int
    latency_ms: float

# Dependency injection — client created once, shared across requests
async def get_openai_client() -> AsyncOpenAI:
    return AsyncOpenAI(api_key=settings.openai_api_key)
```

```
@app.post('/extract', response_model=DocumentResponse)
async def extract_document(
    req: DocumentRequest,
    client: AsyncOpenAI = Depends(get_openai_client),
):
    response = await client.chat.completions.create(
        model=req.model,
        messages=[{'role': 'user', 'content': req.content}],
    )
    return DocumentResponse(...)
```

Streaming from FastAPI to Node.js

- Use StreamingResponse with a generator function
- Yield bytes (b'data: token\n\n') for SSE format
- Node.js backend receives the stream and forwards it to the client
- Use httpx.AsyncClient with stream=True in Node.js ← Python calls

> ⍰ The standard pattern: OpenAI → FastAPI (Python) → Fastify (Node.js) → Next.js (React). Each layer forwards the stream. The FastAPI layer adds token counting, caching, and cost tracking before forwarding.

7.3 — Stop Conditions

☐ Build a FastAPI service with 5 endpoints — all documented automatically at /docs
☐ Pydantic validation: invalid requests return 422 with clear error messages — not 500
☐ Dependency injection: OpenAI client, database session, and config injected via Depends()
☐ Streaming endpoint: FastAPI streams OpenAI tokens to the caller using StreamingResponse
☐ Testing: all endpoints tested with pytest + httpx.AsyncClient — no real OpenAI calls
☐ Docker: FastAPI service runs in a container, accessible from Node.js at http://fastapi:8000
☐ Deployed: FastAPI service live on Railway — Fastify backend can call it via HTTP

TOPIC 8 — Async Python (asyncio, httpx, aiofiles) · *Week 7 · LLM calls are I/O-bound — async is essential*

Calling LLM APIs is slow — 1–10 seconds per request. If your pipeline processes 1000 documents synchronously, it takes hours. With async Python, you can make 50 concurrent LLM calls simultaneously and cut that time to minutes. This is the most important performance skill for AI pipeline engineering.

8.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Video	Python Async Await — Full Tutorial (asyncio explained)	ArjanCodes	YouTube → Search 'Python Async Await Full Tutorial asyncio ArjanCodes'	The best async Python tutorial. ArjanCodes explains coroutines, tasks, and event loop clearly.
2	Video	Python asyncio — The Complete Guide (1.5 hrs)	Tech With Tim	YouTube → Search 'Python asyncio Complete Guide Tech With Tim'	Comprehensive asyncio tutorial covering async/await, gather, tasks, queues.
3	Video	AsyncIO in Python — A Complete Walkthrough	Real Python (YouTube channel)	YouTube → Search 'AsyncIO Python Complete Walkthrough Real Python'	Real Python's clear explanation of the event loop, coroutines, and when NOT to use async.
4	Video	httpx — The Best Python HTTP Client (async support)	ArjanCodes	YouTube → Search 'httpx Best Python HTTP Client ArjanCodes'	httpx is the requests replacement with async support — use for calling LLM APIs.
5	Video	Python Concurrency — asyncio vs Threading vs Multiprocessing	ArjanCodes	YouTube → Search 'Python Concurrency asyncio Threading Multiprocessing ArjanCodes'	Critical decision guide: when to use async (I/O), threads (legacy libs), multiprocessing (CPU).
6	Video	asyncio.gather vs asyncio.TaskGroup — Concurrent LLM Calls	mCoding	YouTube → Search 'asyncio gather TaskGroup Concurrent mCoding'	TaskGroup (Python 3.11+) is the modern, safer way to run concurrent async tasks.
7	Video	Semaphore in Python asyncio — Rate Limiting Concurrent Calls	Tech With Tim	YouTube → Search 'Semaphore Python asyncio Rate Limiting Tech With Tim'	Semaphore to limit to 10 concurrent LLM calls — prevents 429 rate limit errors.
8	Video	aiofiles — Async File I/O in Python	Coding Entrepreneurs	YouTube → Search 'aiofiles Async File IO Python Coding Entrepreneurs'	aiofiles for reading/writing large documents without blocking the event loop.

8.2 — What to Study

async/await Fundamentals

- Coroutines: async def — functions that can be paused and resumed
- Awaiting: await — suspend the coroutine until the I/O operation completes
- Event loop: runs coroutines and switches between them during I/O waits
- asyncio.run() — the entry point for running async code from synchronous code
- Key insight: async Python uses ONE thread — it is NOT multi-threading

Concurrent LLM Calls — The Core Pattern

```
# Process 1000 documents with 20 concurrent LLM calls
import asyncio
from openai import AsyncOpenAI

async def embed_document(client: AsyncOpenAI, text: str) -> list[float]:
    response = await client.embeddings.create(
        input=text, model='text-embedding-3-small'
    )
    return response.data[0].embedding

async def process_all_documents(documents: list[str]) -> list[list[float]]:
    client = AsyncOpenAI()
    semaphore = asyncio.Semaphore(20)   # max 20 concurrent calls

    async def embed_with_limit(text: str) -> list[float]:
        async with semaphore:
            return await embed_document(client, text)

    # Process all documents concurrently (with semaphore limit)
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(embed_with_limit(doc)) for doc in documents]

    return [task.result() for task in tasks]

# 1000 docs, 20 concurrent → ~50x faster than sequential
# Sequential: 1000 × 0.5s = 500 seconds
# Concurrent: 1000 / 20 × 0.5s = 25 seconds
```

Rate Limiting with Semaphore + Tenacity Retry

- asyncio.Semaphore(N): limit to N concurrent async operations
- @retry(wait=wait_exponential(), retry=retry_if_exception_type(RateLimitError))
- Combine: semaphore prevents hitting rate limits, tenacity handles when you do
- asyncio.sleep(0) inside tight loops — yield control to the event loop

> ⍰ The magic number for OpenAI embeddings API: Semaphore(50) for text-embedding-3-small. You can push 50 concurrent embedding calls before hitting rate limits on the default tier. For GPT-4o: Semaphore(5).

8.3 — Stop Conditions

☐ Embed 1000 documents concurrently with asyncio.Semaphore(20) — time it vs sequential (prove 10x+ speedup)
☐ Use asyncio.TaskGroup to process documents concurrently with proper error propagation
☐ Implement retry with tenacity: exponential backoff on OpenAI 429 errors — tested with mock
☐ Explain when to use async vs threads vs multiprocessing — give a correct example of each
☐ Use httpx.AsyncClient to call your FastAPI service from a Python script concurrently
☐ Read 500 files concurrently using aiofiles — verify no event loop blocking

TOPIC 9 — Data Pipelines with Prefect · *Week 8 · Reliable, observable, scheduled AI pipelines*

A data pipeline that runs once is a script. A data pipeline that runs reliably on a schedule, retries on failure, alerts on errors, and has a monitoring dashboard is a production system. Prefect is the modern pipeline orchestrator that makes this straightforward. In Phase 7 you will go deeper into Airflow — for now, Prefect handles everything you need.

9.1 — Exact Video Resources

#	Type	Exact Title / Playlist Name	Channel Name	Where to Find	Notes / Why This Resource
1	Course	Prefect 2.0 Tutorial — Getting Started (Official, 5-part series)	Prefect (official channel)	YouTube → Search 'Prefect 2.0 Tutorial Getting Started Prefect' → open playlist	Official Prefect tutorial series. Build a real pipeline from scratch. Watch all 5 parts.
2	Video	Prefect vs Airflow vs Luigi — Which Should You Use?	Data with Marc	YouTube → Search 'Prefect vs Airflow vs Luigi Which Should You Use Data with Marc'	Explains when to use Prefect over Airflow. Sets context before diving into Prefect.
3	Video	Build a Data Pipeline with Prefect (Real World Example)	Zach Wilson	YouTube → Search 'Build Data Pipeline Prefect Real World Zach Wilson'	Zach Wilson is the best data engineering YouTuber. Real-world Prefect pipeline.
4	Video	Prefect Deployments and Scheduling — Full Guide	Prefect (official channel)	YouTube → Search 'Prefect Deployments Scheduling Full Guide Prefect official'	Schedule your AI ingestion pipeline to run daily/hourly — essential for production.
5	Video	Prefect Notifications and Alerts — Slack and Email	Prefect (official channel)	YouTube → Search 'Prefect Notifications Alerts Slack Email Prefect official'	Get Slack notifications when your AI pipeline fails — production monitoring.
6	Video	ETL Pipeline Python — Extract Transform Load Tutorial	Tech With Tim	YouTube → Search 'ETL Pipeline Python Extract Transform Load Tutorial Tech With Tim'	ETL fundamentals in Python — complements Prefect with the conceptual foundation.
7	Docs	Prefect Official Documentation	Prefect Team	docs.prefect.io — read: Tutorial, Concepts, Deployments, Notifications	Excellent docs. Read Tutorial + Concepts completely. Prefect's docs are beginner-friendly.

9.2 — What to Study

Prefect Core Concepts

- @task: decorates a Python function — adds retries, caching, logging, observability
- @flow: orchestrates tasks — the top-level unit of a pipeline
- Task run states: Pending → Running → Completed / Failed / Retrying
- Retries: @task(retries=3, retry_delay_seconds=10) — automatic retry with delay
- Caching: @task(cache_key_fn=..., cache_expiration=timedelta(hours=24)) — skip re-processing

- Deployments: run flows on a schedule or trigger them via API

AI Document Ingestion Pipeline with Prefect

```
from prefect import task, flow
from prefect.tasks import task_input_hash
from datetime import timedelta

@task(retries=3, retry_delay_seconds=30, log_prints=True)
def fetch_documents(source_url: str) -> list[dict]:
    """Fetch documents from external source."""
    # ... fetch logic
    print(f'Fetched {len(docs)} documents')
    return docs

@task(cache_key_fn=task_input_hash,       # cache by input hash
      cache_expiration=timedelta(days=1)) # don't re-embed same text
def embed_document(text: str) -> list[float]:
    """Generate embedding — cached to avoid duplicate API calls."""
    return openai.embeddings.create(input=text, model='text-embedding-3-small')
           .data[0].embedding

@task(retries=2)
def store_in_vector_db(chunks: list[dict]) -> None:
    """Store chunks and embeddings in Qdrant."""
    # ... upsert to Qdrant

@flow(name='AI Document Ingestion', log_prints=True)
def ingest_documents(source_url: str) -> None:
    docs = fetch_documents(source_url)
    for doc in docs:
        chunks = parse_and_chunk(doc)    # another @task
        embeddings = [embed_document(c['text']) for c in chunks]
        store_in_vector_db(chunks)
    print('Pipeline completed successfully')
```

Monitoring & Alerts

- Prefect Cloud UI: every flow run visible with timing, logs, and task states
- Notifications: Slack alert when pipeline fails — '@team Pipeline failed!'
- Artifacts: store charts and metrics as flow artifacts for easy debugging
- Flow run health: track success rate, average duration, failure patterns

9.3 — Stop Conditions

☐ Build a 5-task Prefect flow: fetch → parse → chunk → embed → store — all with retries
☐ Add task caching: re-running the flow skips embedding for already-processed documents

☐ Schedule the flow to run daily at 2am UTC — verify it runs automatically
☐ Add Slack notification: pipeline completion and failure both send Slack messages
☐ Monitor in Prefect Cloud: all flow runs visible with task-level timing and logs
☐ Process 500 documents through the pipeline — zero data loss on simulated network errors

📅 Phase 3 — Week-by-Week Schedule (12 Weeks)

Follow this schedule week by week. The first 4 weeks are pure Python learning — no AI libraries yet. The last 4 weeks integrate everything into the capstone project pipeline.

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 1	Python Fundamentals	Python Full Course — Mosh (first 2.5 hrs) Python in 100 Seconds — Fireship Python Decorators — Corey Schafer	Rewrite 3 Node.js utility functions in Python Implement @retry decorator from scratch	Python syntax fluent; can write list comprehensions, classes, and decorators
Wk 2	Python Advanced + Environment Setup	Intermediate Python — Tech With Tim (Ep 1–10) Poetry Full Tutorial — ArjanCodes Python Project Setup 2024 — ArjanCodes	Set up Poetry project with Ruff + mypy Write 5 Pythonic patterns: generators, context managers	Poetry project runs cleanly; mypy passes; ruff finds zero lint errors
Wk 3	NumPy	NumPy Full Tutorial — Keith Galli (all) Linear Algebra for ML — freeCodeCamp (first 1.5 hrs) Cosine Similarity Python — NeuralNine	Implement cosine similarity from scratch Build naive vector search over 10K fake embeddings	Can compute similarity of embedding vectors without sklearn; shapes are correct
Wk 4	Pandas + Data Viz	Pandas Full Course — Rob Mulla (all) Pandas Data Cleaning — Keith Galli Matplotlib Tutorial — Keith Galli (first 1.5 hrs)	Clean a messy 10,000-row document dataset Build 4-panel chart: token dist, category counts, scatter, time series	Dataset fully cleaned; charts render correctly; no loops in Pandas code
Wk 5	File Parsing — PDF, DOCX, HTML	PyMuPDF Tutorial — Nicholas Renotte Python-docx Tutorial — Tech With Tim Beautiful Soup 4 Tutorial — Tech With Tim	Build universal parser: PDF + DOCX + HTML → clean text + metadata Test on 20 real documents	Parser handles all 3 formats; metadata extracted correctly; errors handled gracefully
Wk 6	FastAPI Basics + Pydantic	FastAPI Tutorial — ArjanCodes (all) Pydantic v2 Complete Guide — ArjanCodes FastAPI Dependency Injection — ArjanCodes	Build FastAPI service with 3 endpoints: /parse, /extract, /health Pydantic models for all inputs/outputs	API documented at /docs; invalid inputs return 422; DI working
Wk 7	Async Python + FastAPI Streaming	Python Async Await — ArjanCodes Python Concurrency — ArjanCodes FastAPI Streaming Responses — Pixegami Semaphore asyncio — Tech With Tim	Process 100 docs with asyncio.Semaphore(20) — measure speedup vs sequential Add streaming endpoint to FastAPI	Concurrent processing 10x faster than sequential; streaming endpoint works
Wk 8	Prefect Pipelines	Prefect 2.0 Tutorial (5-part official series) Build Data Pipeline Prefect — Zach Wilson Prefect Deployments — Prefect official	Build 5-stage document ingestion pipeline with Prefect Schedule it daily; add Slack alerts	Pipeline runs on schedule; failures send Slack notification; Prefect Cloud shows all runs
Wk 9	Integration: Connect Python to Node.js Backend	FastAPI Docker Deployment — Tech With Tim Docker for Python Developers — Tech With Tim	Dockerise FastAPI service Connect Fastify backend to FastAPI via HTTP — test end-to-end	Fastify can call FastAPI; Docker Compose runs both services together
Wk 10	Testing + Error Handling	FastAPI Testing — ArjanCodes Python Testing with pytest — Corey Schafer (playlist Ep 1–5)	Write pytest tests for all FastAPI endpoints Mock all OpenAI and external API calls	80%+ test coverage; CI passes without real API keys

Week	Topic Focus	Exact Videos to Watch	Hands-On Task	Done When...
Wk 11	Capstone Build — Core Pipeline	Review: Unstructured.io tutorial + Prefect deployments	Build the full Document Intelligence Pipeline PDF upload → parse → chunk → embed → store	End-to-end pipeline working; 20 test documents processed correctly
Wk 12	Capstone Polish + Deploy + Open Source	Railway deployment docs Docker Compose multi-service guide	Deploy FastAPI to Railway Connect to Phase 2 backend Write detailed README with architecture diagram	Live API on Railway; GitHub repo public with one-command setup; detailed README

⍰ Phase 3 Capstone — AI Document Intelligence Pipeline

This is Portfolio Project #3. It is the backend AI brain that powers every RAG system you will build in Phase 4 and 5. Open-source it on GitHub — practical AI data engineering tools attract serious attention from AI engineering hiring managers.

Full Project Specification

Project Name	ai-document-intelligence (GitHub repo name)
Core Language	Python 3.11+ with FastAPI + asyncio + Pydantic v2
Document Support	PDF (PyMuPDF), DOCX (python-docx), HTML (BeautifulSoup4), TXT, CSV, JSONL
AI Integration	OpenAI Embeddings API (text-embedding-3-small), GPT-4o structured extraction
Vector Storage	Qdrant (self-hosted via Docker) for embedding storage and similarity search
Pipeline Engine	Prefect 2.0 — scheduled ingestion flows with retries, caching, and alerts
Async Processing	asyncio.Semaphore(20) for concurrent LLM calls — 10-50x faster than sequential
Data Processing	Pandas for metadata management, NumPy for embedding operations
Integration	FastAPI REST API called by Phase 2 Fastify backend — full stack working together
Deployment	Docker Compose (local dev) + Railway (production) + Prefect Cloud (pipeline)

Required Features — Build All of These

Document Ingestion API

- POST /api/v1/documents — upload file (multipart), returns job_id immediately
- GET /api/v1/documents/{job_id}/status — poll for processing status + progress %
- GET /api/v1/documents/{doc_id} — retrieve parsed document + metadata
- DELETE /api/v1/documents/{doc_id} — remove document and all associated chunks/embeddings
- GET /api/v1/documents — list all documents for an organisation (paginated)

Processing Pipeline (5 stages, each a Prefect @task)

- Stage 1 — Validate: check file type, size (<50MB), and content (not empty/corrupt)
- Stage 2 — Parse: extract clean text + metadata using universal parser
- Stage 3 — Chunk: sliding window chunking (512 tokens, 50 overlap) with metadata tracking
- Stage 4 — Embed: concurrent embedding generation with asyncio.Semaphore(20)
- Stage 5 — Store: upsert chunks + vectors to Qdrant with metadata payload

AI Extraction Feature

- POST /api/v1/extract — send document content + JSON schema → returns structured data
- Uses OpenAI structured outputs (response_format=json_schema) for reliable extraction
- Example schemas: invoice extractor (vendor, amount, date), contract parser (parties, terms)
- Extraction results stored in PostgreSQL as JSONB — queryable metadata

Similarity Search API (Preview for Phase 4 RAG)

- POST /api/v1/search — query text → embedding → Qdrant search → top-K chunks
- Metadata filtering: search within specific document type, date range, category
- Returns: ranked chunks with similarity score, source document, page number

Observability & Monitoring

- Structured logging: every pipeline stage logs timing, token counts, and errors
- Prefect Cloud dashboard: all pipeline runs visible with task-level timing
- Metrics endpoint: GET /metrics — exposes processing stats as JSON
- Slack alerts: pipeline failures notify #ai-pipeline-alerts channel

GitHub README Must Include

- Architecture diagram: Node.js → FastAPI → Prefect → Qdrant flow
- Supported formats table with parsing library used for each
- Benchmark results: documents/minute at different concurrency levels
- One-command local setup: 'docker compose up' starts FastAPI + Qdrant + Prefect
- Example API calls with curl snippets for each endpoint
- 'How it works' section explaining the 5-stage pipeline in plain English

Full System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          CLIENT LAYER                                 │
│  Next.js Frontend (Vercel)  ⟷  Fastify API (Railway, Phase 2)        │
└──────────────────────────┬──────────────────────────────────────────┘
                           │  HTTP calls (httpx/fetch)
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│                 FastAPI Service (Railway / Docker)                    │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  Routers: /documents  /extract  /search  /health  /metrics   │    │
│  │  Pydantic v2 validation → Dependency injection               │    │
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
      │                    │                      │
      ▼                    ▼                      ▼
  Prefect Flow         PostgreSQL             OpenAI API
  ┌──────────┐         (Metadata +           (Embeddings +
  │ @task 1  │          Results)              Extraction)
  │ Validate │
```

```
    | @task 2  |
    | Parse    |──→ PyMuPDF / python-docx / BeautifulSoup4
    | @task 3  |
    | Chunk    |──→ Sliding window (512 tok, 50 overlap)
    | @task 4  |
    | Embed    |──→ asyncio.Semaphore(20) concurrent calls
    | @task 5  |
    | Store    |──→ Qdrant (Docker / Railway)
    └──────────┘
   Prefect Cloud (monitoring, scheduling, alerts)
```

Estimated Time Investment

Weeks 1–2 (Python Fundamentals + Env Setup)	~20 hrs — 12 hrs videos + 8 hrs coding exercises
Week 3 (NumPy)	~14 hrs — 8 hrs videos + 6 hrs exercises
Week 4 (Pandas + Visualisation)	~18 hrs — 10 hrs videos + 8 hrs data cleaning project
Weeks 5–6 (File Parsing + FastAPI)	~22 hrs — 10 hrs videos + 12 hrs building parser + API
Week 7 (Async Python)	~14 hrs — 8 hrs videos + 6 hrs async pipeline
Week 8 (Prefect Pipelines)	~14 hrs — 6 hrs videos + 8 hrs pipeline build
Weeks 9–10 (Integration + Testing)	~16 hrs — 4 hrs videos + 12 hrs coding + testing
Weeks 11–12 (Capstone + Deploy)	~20 hrs — building, deploying, documenting
Total Phase 3	~138 hours over 12 weeks ≈ 2.5 hrs/weekday + 5 hrs/weekend

Phase 3 Completion Check: You have completed Phase 3 when: (1) The Document Intelligence Pipeline is live on Railway with working API endpoints, (2) Docker Compose starts the full stack in <60 seconds, (3) The Fastify backend from Phase 2 can successfully call the FastAPI service, (4) The GitHub README clearly documents the architecture and setup steps. At this point you are genuinely a Python AI engineer.

The single most common Phase 3 mistake: spending all 12 weeks on NumPy and Pandas theory and never building the FastAPI service. Pandas and NumPy are reference knowledge — you look them up when needed. FastAPI and async Python are skills you must build deeply. Do not get stuck on data science tutorials.